

1. CRUD Operations

CRUD stands for Create, Read, Update, and Delete—the four basic functions of persistent storage.

* Create (INSERT): Adds new rows to a table.

SQL

```
INSERT INTO employees (id, name, department, salary)
VALUES (1, 'Alice', 'Engineering', 95000);
```

* **Read (SELECT):** Retrieves data from a table.

```sql

```
SELECT name, salary FROM employees WHERE department = 'Engineering';
```

\* Update (UPDATE): Modifies existing data. Always use a WHERE clause, or you will update every row!

SQL

```
UPDATE employees SET salary = 100000 WHERE id = 1;
```

\* \*\*Delete (DELETE):\*\* Removes rows. \*Always use a `WHERE` clause, or you will wipe the table clean!\*

```sql

```
DELETE FROM employees WHERE id = 1;
```

2. Constraints

Constraints are rules applied to table columns to enforce data integrity and accuracy.

* NOT NULL: Ensures a column cannot have a NULL value.

* UNIQUE: Ensures all values in a column are distinct.

* PRIMARY KEY: A combination of NOT NULL and UNIQUE. Uniquely identifies each row in a table.

* FOREIGN KEY: Prevents actions that would destroy links between tables (enforces referential integrity).

* CHECK: Ensures that the values in a column satisfy a specific condition (e.g., CHECK (age >= 18)).

* DEFAULT: Sets a default value for a column if no value is specified.

3. Clauses: Order By, Group By, Having

These clauses help filter, sort, and slice your data.

The Execution Order Cheat Sheet

When writing code, you write it in one order, but SQL executes it in this order:

```
FROM $\\rightarrow$ WHERE $\\rightarrow$ GROUP BY $\\rightarrow$ HAVING
$\\rightarrow$ SELECT $\\rightarrow$ ORDER BY
```

* ORDER BY: Sorts the result set in ascending (ASC, default) or descending (DESC) order.

SQL

```
SELECT name, salary FROM employees ORDER BY salary DESC;
```

* **`GROUP BY`**:

 Groups rows that have the same values into summary rows (often used with aggregate functions).

```sql

```
SELECT department, COUNT(*) FROM employees GROUP BY department;
```

\* HAVING: Identical to WHERE, but exclusively used for filtering grouped data and aggregate functions (because WHERE cannot filter on aggregates).

SQL

```
SELECT department, AVG(salary)
FROM employees
GROUP BY department
HAVING AVG(salary) > 80000;
```

---

## ## 4. Aggregate Functions

Aggregate functions perform a calculation on a set of values and return a single value. They completely ignore `NULL` values (except for `COUNT(\*)`).

| Function | Description                                   | Example                                           |
|----------|-----------------------------------------------|---------------------------------------------------|
| Count    | Returns the number of rows                    | <code>`SELECT COUNT(*) FROM employees;`</code>    |
| Sum      | Returns the total sum of a numeric column     | <code>`SELECT SUM(salary) FROM employees;`</code> |
| Avg      | Returns the average value of a numeric column | <code>`SELECT AVG(salary) FROM employees;`</code> |
| Min      | Returns the smallest value                    | <code>`SELECT MIN(salary) FROM employees;`</code> |
| Max      | Returns the largest value                     | <code>`SELECT MAX(salary) FROM employees;`</code> |

---

## ## 5. Joins

Joins are used to combine rows from two or more tables based on a related column between them.

| INNER JOIN                 | LEFT JOIN                  | RIGHT JOIN                 |
|----------------------------|----------------------------|----------------------------|
| <code>[ A ( ? ) B ]</code> | <code>[ ( A ? ) B ]</code> | <code>[ A ( ? B ) ]</code> |

Only matching rows All A + matching B All B + matching A

\* `**`INNER JOIN`**`: Returns records that have matching values in both tables.

```
```sql
```

```
SELECT employees.name, departments.dept_name
FROM employees
```

```
INNER JOIN departments ON employees.dept_id = departments.id;
```

* `LEFT (OUTER) JOIN`: Returns all records from the left table, and the matched records from the right table. If no match is found, NULL results from the right side.

* `RIGHT (OUTER) JOIN`: Returns all records from the right table, and the matched records from the left table.

* `FULL (OUTER) JOIN`: Returns all records when there is a match in either left or right table. (Note: MySQL doesn't natively support this; you mimic it using a UNION of a LEFT and RIGHT join).